COMP-3670 Project

In this project you will implement a streaming video server and client that communicate using the Real-Time Streaming Protocol (RTSP) and send data using the Real-time Transfer Protocol (RTP). Your task is to implement the RTSP protocol in the client and implement the RTP packetization in the server.

We will provide you with code that implements the RTSP protocol in the server, the RTP de-packetization in the client, and takes care of displaying the transmitted video.

Do not worry if you have never used Python before! This project requires no complex Python knowledge! The focus of this project is implementing concepts discussed in class and is not meant to require a lot of coding. In fact, this project requires you to only write 13 lines of code!

Code

Client.py
The client which you use to send RTSP commands and displays the video.

Server.py
The server which responds to the RTSP requests and streams back the video.

Complete.py
This file contains the two methods you need to complete for the project.
"rtsp_payload" is used to build the RTSP payload which the client sends to the server.
"rtp_header" is used to construct the header for RTP messages which the server sends to the client.
You do not need to import any modules into "Complete.py" to complete the assignment as there are no complex data types or methods you need to access, only strings and bytes. Do not change the method names or parameters.

Running the code

After completing the code, you can run it as follows. **See the setup section towards the end of the document for a quick getting up and running.**

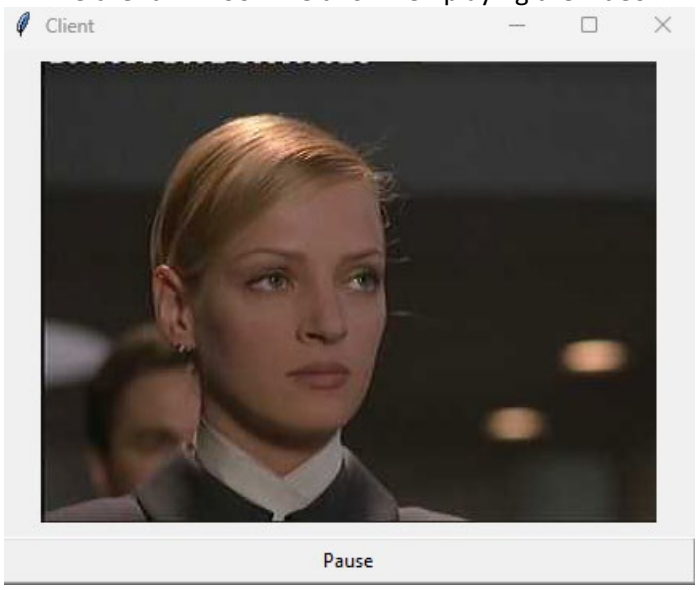
First, start the server with the command:
python Server.py server_port

- server_port is the port your server listens to for incoming RTSP connections. The standard RTSP port is 554, but you will need to choose a port number greater than 1024. By default, this is 7777.

Then, start the client with the command:
python Client.py server_address server_port rtp_port video_file

- server_address is the address of the machine where the server is running. By default, this is your local machine being "127.0.0.1".
- server_port is the port where the server is listening on. By default, this is 7777.
- rtp_port is the port where the RTP packets are received. By default, this is 7778.
- video_file is the name of the video file you want to request. A file "movie.Mjpeg" has been provided and is the default value. The file format is described in Appendix section.

The client will look like this when playing the video:



Some RTSP commands happen automatically, and some can be controlled with a button. A normal RTSP interaction goes as follows:

1. Once launched, the client sends SETUP (integer value of 0). This command is used to set up the session and transport parameters.
2. The client sends PLAY (integer value of 1) after the setup is complete. This command starts the playback.
3. The client may send PAUSE (integer value of 2) if it wants to pause during playback by pressing the button. This button turns into a play button to send the PLAY command upon pausing the video.
4. The client sends TEARDOWN (integer value of 3) when closing the application. This command terminates the session and closes the connection.

The server always replies to all the messages that the client sends. The code 200 means that the request was successful while the codes 404 and 500 represent FILE_NOT_FOUND error and connection error respectively. In this project, you do not need to implement any other reply codes. For more information about RTSP, please see RFC 2326.

The Client

Your first task is to implement the RTSP protocol on the client side. To do this, you need to complete the "rtsp_payload" method in "Complete.py". You will need to implement the actions for the above request types: SETUP, PLAY, PAUSE, and TEARDOWN. Packets must be formatted as follows where all values are integers except for *MOVIE FILE* which is a string.

SEQUENCE NUMBER|*COMMAND*

SETUP commands must also add the movie to stream and the RTP port, so their data packet looks like:
SEQUENCE NUMBER|*COMMAND*|*MOVIE FILE* |*RTP PORT*

Note that all these are delimited by a "|", and they must be in this order. Redundant data should not be sent, so sending the movie file or RTP port with any command other than SETUP will lose marks.

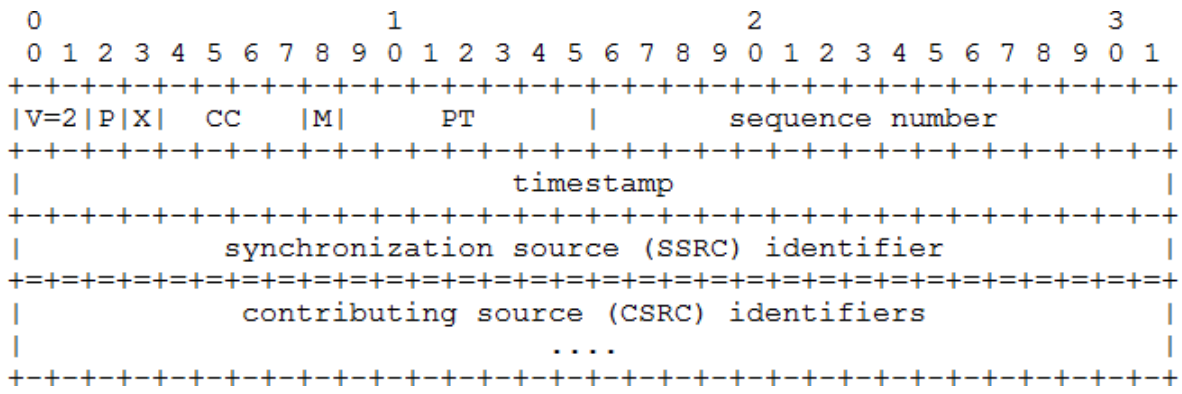
The Server

On the server side, you will need to implement the "rtp_header" method in "Complete.py" for the packetization of the video data into RTP packets. You need to set the fields in the packet header.

When the server receives the PLAY-request from the client, the server reads one video frame from the file and creates an RtpPacket-object which is the RTP-encapsulation of the video frame. It then sends the frame to the client over UDP every 50 milliseconds.

For the encapsulation, the server calls the encode function for the header of the packet. Your task is to write this function. You will need to do the following: (the letters in parenthesis refer to the fields in the RTP packet format below).

- Set the RTP-version field (V).
- Set padding (P), extension (X), number of contributing sources (CC), and marker (M) fields. These are all set to zero in this project.
- Set payload type field (PT).
- Set the sequence number.
- Set the timestamp.
- Set the source identifier (SSRC). This field identifies the server.
- Because we have no other contributing sources (field CC == 0), the CSRC-field does not exist. The length of the packet header is therefore 12 bytes, or the first three lines from the diagram below.
- More details on each of the fields can be found in the comments of the code, which are based upon: https://en.wikipedia.org/wiki/Real-time_Transport_Protocol



The above diagram is in the network byte order (also known as big-endian). Python uses the same byte order, so you do not need to transform your packet header into the network byte order.
For more details on RTP, please see RFC 1889.

Important note and hint: The way we pack bits is the opposite of how we unpack them! This happens in the "listen_rtp" method in "Client.py"! On line 127, you can see the sequence number being unpacked. This is a great starting point for you! However, you may notice none of the other header fields are unpacked. This is because we can't give everything away! In a real application, you should verify all header fields, but they have been left out here for learning purposes. This does mean; however, your video will play even if you only get the sequence number correct in the header. **You do however still need to finish the rest of the header for your submission which we will detail later.**

Twiddling the Bits

Here are some examples on how to set and check individual bits or groups of bits. Note that in the RTP packet header format smaller bit-numbers refer to higher order bits, that is, bit number 0 of a byte is 2^7 and bit number 7 is 1 (or 2^0). In the examples below, the bit numbers refer to the numbers in the above diagram.

Because the header-field of the RtpPacket class is of type bytearray, you will need to set the header one byte at a time, that is, in groups of 8 bits. The first byte has bits 0-7, the second byte has bits 8-15, and so on.

To set bit number n in variable mybyte of type byte:
mybyte = mybyte | 1 << (7 - n)

To set bits n and n + 1 to the value of foo in variable mybyte:
mybyte = mybyte | foo << (7 - n)

Note that foo must have a value that can be expressed with 2 bits, that is, 0, 1, 2, or

To copy a 16-bit integer foo into 2 bytes, b1 and b2:
b1 = (foo >> 8) & 0xFF
b2 = foo & 0xFF

After this, b1 will have the 8 high-order bits of foo and b2 will have the 8 low-order bits of foo. You can copy a 32-bit integer into 4 bytes in a similar way.

Bit Example

Suppose we want to fill in the first byte of the RTP packet header with the following values:
V = 2
P = 0
X = 0
CC = 3

In binary this would be represented as
1 0 | 0 | 0 | 0 0 1 1
V=2 P X CC = 3
2^7……..2^0

Submission and Grading

An automatic grader has been set up for this project. Run your code on https://comp3670.stevenrice.ca where you will be given your score. There are a few conditions you need to meet for your code to run:

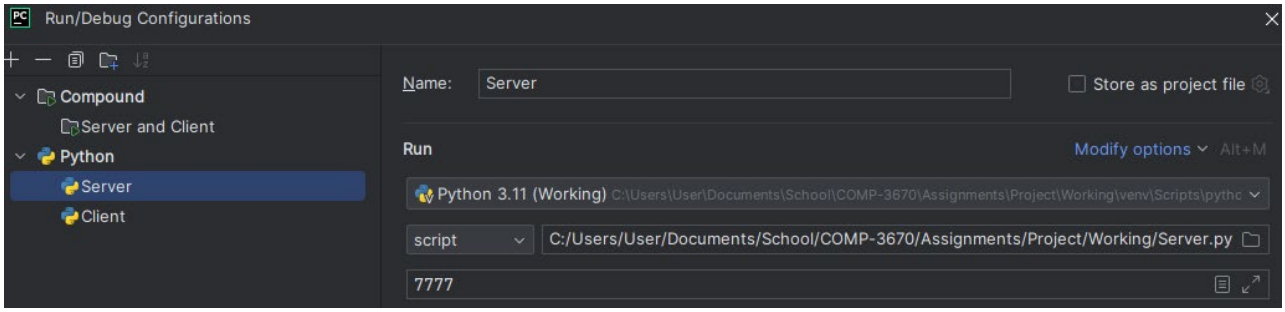
1. Do not include the method definition in your submission.
2. Do not change variable names.
3. You cannot make calls to other methods.
4. You cannot import any modules.
5. The solution does not require any while/for loops, so including any will not run the code.
6. Remove any print statements you included for debugging.
7. Single-line comments (starting with #) are okay!
8. **Note that the grading server goes to sleep after not being used for 15 minutes, so if it takes a minute or two to load, you are likely the first one to use it in a little.**
9. **If you encounter any unexpected errors, or you believe the code you are submitting is valid, but you are getting no results from it, please contact rice118@uwindsor.ca with the details that caused the error.**

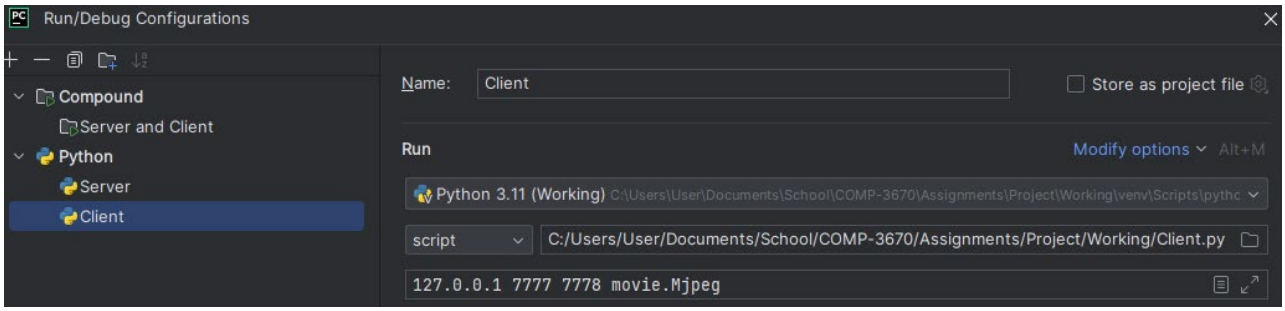
The project is out of 20 marks. “rtsp_payload” is worth 8 marks and “rtp_header” is worth 12 marks. “rtsp_payload” will be graded by ensuring that the format matches that described in “the client” section, deducting marks for missing, extra, or the wrong order of values. “rtp_header” will be graded by comparing bits with expected values. Each method is tested on 100 random test cases. For the “rtsp_payload” test cases, the sequence number, move file name, and RTP port will be randomized. There will be an even amount of test cases for each command type being SETUP, PLAY, PAUSE, and TEARDOWN. “rtp_header” will have random values generated within each field’s data size. For example, the version field can be a maximum of 2 bytes, so a random value in the range $[0, 2^2)$ will be generated. The CC and extension fields are always 0 in the test cases. To help you debug, if your score is not perfect, the details of the first test case which failed will be output along with the variables, your answer, and the expected answer. **After running your code, be sure to submit your “Complete.py” file on Brightspace in case we need to inspect your solution.**

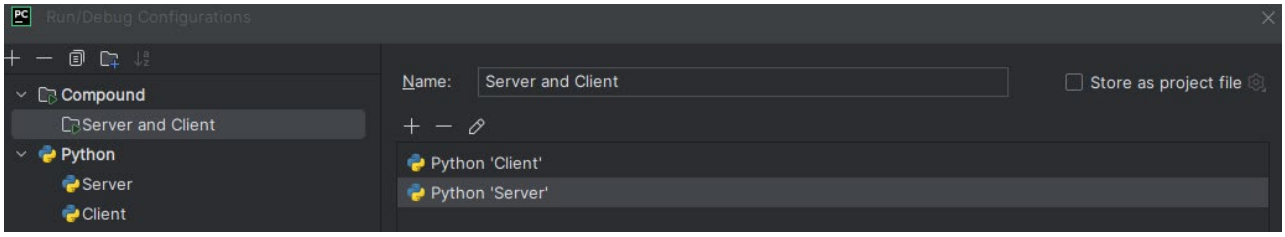
Setup

Even if you have never used Python before, these few steps will get your project environment up and running quickly.

1. Download and install Python: https://www.python.org/downloads
2. There are plenty of IDEs available, but as a student, you have access to the entire JetBrains suite of tools for free, including their PyCharm IDE. You can get access here: https://www.jetbrains.com/shop/eform/students
3. Open the provided files in your IDE of choice. If using PyCharm, you will be prompted to select a Python interpreter on your first time opening a project. If you installed Python to a standard location, you should see it as an option. Otherwise, you may need to search for your Python installation directory. If using PyCharm, this will also guide you through setting up a Python virtual environment. Although entirely optional, especially if this is the only Python project you are working on, virtual environments allow you to install packages on a per-project basis to ensure there are no version conflicts between multiple projects.
4. Your IDE will likely indicate that there are errors regarding the "PIL" imports as you need the pillow package. **Note that you need the package "pillow", not "PIL". "PIL" is deprecated, and "pillow" is its successor, using its same import name to ensure compatibility.** You have a few options to install it. If you wish to do so by the command line, you could run "pip install pillow". However, if you are using PyCharm, you should be able to hover over the import error and it will prompt you to ask if you would like to install "pillow". Note that it may also offer to install "PIL", and again, install "pillow", not "PIL".
5. To run the project, you must run the server and the client. You may try with different parameters if you wish, but some simple settings for each to get up and running are:
    a. **Note that if not passed, default values as detailed in the "running the code" section will be used for the arguments.**
    b. python Server.py 7777
    c. python Client.py 127.0.0.1 7777 7778 movie.Mjpeg
    d. In PyCharm, you can easily launch both projects together with a single click.
        i. In the top right, you should see a dropdown arrow. If you have not changed any settings, by default, it should say "Current File". Click this dropdown and select "Edit Configurations…".
        ii. In the top left corner, hit the "+" icon and select "Python". Name this configuration "Server" and in the script field navigate to and select your "Server.py" file. In the parameters field, input "7777".
        iii. Using the "+" icon again add a "Python" option and set it up for the client. Name it "Client", select "Client.py", and add the parameters "127.0.0.1 7777 7778 movie.Mjpeg".
        iv. Once again go back to the "+" icon and select "Compound". Name the compound launch "Server and Client". There will be another "+" icon inside the compound's settings. From this, add the server and client configurations you just made.
        v. Apply changes and close the window. In the top right arrow dropdown, you initially used, if it did not already change to do so, choose to use the "Server and Client" configuration you just made. Now, to run your code, simply hit the play button. If you wish to debug using breakpoints, use the green bug button right beside the play button.
        vi. **You may need to set the working directory for the Server and Client configurations, which is simply the directory all the files for this project are in.**
        vii. The setups for PyCharm can be seen on the next page.


Run/Debug Configurations
Compound
Server and Client
Python
Server
Client
Name:
Server
Store as project file
Run
Modify options
Alt+M
Python 3.11 (Working) C:\Users\User\Documents\School\COMP-3670\Assignments\Project\Working\venv\Scripts\pythc
script
C:/Users/User/Documents/School/COMP-3670/Assignments/Project/Working/Server.py
7777


Run/Debug Configurations
Compound
Server and Client
Python
Server
Client
Name:
Client
Store as project file
Run
Modify options
Alt+M
Python 3.11 (Working) C:\Users\User\Documents\School\COMP-3670\Assignments\Project\Working\venv\Scripts\pythc
script
C:/Users/User/Documents/School/COMP-3670/Assignments/Project/Working/Client.py
127.0.0.1 7777 7778 movie.Mjpeg


Run/Debug Configurations
Compound
Server and Client
Python
Server
Client
Name:
Server and Client
Store as project file
Python 'Client'
Python 'Server'

Sample Run

In this run, the connection is made to set up and automatically play the video. Then, after a moment the pause button is pressed. The play button is then pressed again, followed by another press of the pause button. Finally, the client UI is closed, tearing down the connection.

Client	Server
Data sent: 1|0|movie.Mjpeg|7778	Data received: 1|0|movie.Mjpeg|7778
Data sent: 2|1	Data received: 2|1
RTP Sequence Number Received: 1	RTP Sequence Number Sent: 1
RTP Sequence Number Received: 2	RTP Sequence Number Sent: 2
RTP Sequence Number Received: 3	RTP Sequence Number Sent: 3
RTP Sequence Number Received: 4	RTP Sequence Number Sent: 4
RTP Sequence Number Received: 5	RTP Sequence Number Sent: 5
RTP Sequence Number Received: 6	RTP Sequence Number Sent: 6
RTP Sequence Number Received: 7	RTP Sequence Number Sent: 7
RTP Sequence Number Received: 8	RTP Sequence Number Sent: 8
RTP Sequence Number Received: 9	RTP Sequence Number Sent: 9
RTP Sequence Number Received: 10	RTP Sequence Number Sent: 10
RTP Sequence Number Received: 11	RTP Sequence Number Sent: 11
Data sent: 3|2	Data received: 3|2
Data sent: 4|1	Data received: 4|1
RTP Sequence Number Received: 12	RTP Sequence Number Sent: 12
RTP Sequence Number Received: 13	RTP Sequence Number Sent: 13
RTP Sequence Number Received: 14	RTP Sequence Number Sent: 14
RTP Sequence Number Received: 15	RTP Sequence Number Sent: 15
RTP Sequence Number Received: 16	RTP Sequence Number Sent: 16
RTP Sequence Number Received: 17	RTP Sequence Number Sent: 17
RTP Sequence Number Received: 18	RTP Sequence Number Sent: 18
RTP Sequence Number Received: 19	RTP Sequence Number Sent: 19
Data sent: 5|2	Data received: 5|2
Data sent: 6|3	Data received: 6|3

Appendix

The project's MJPEG (Motion JPEG) format: In this project, the server streams a video which has been encoded into a proprietary MJPEG file format. This format stores the video as concatenated JPEG-encoded images, with each image being preceded by a 5-Byte header which indicates the bit size of the image. The server parses the bitstream of the MJPEG file to extract the JPEG images on the fly. The server sends the images to the client at periodic intervals. The client then displays the individual JPEG images as they arrive from the server.